



Cartograph

Software Design Document — how the system is built, decided component by component before a line of it is written.

OWNER	Jagath Manjunath
DOCUMENT	SDD (deliverable of SDLC Phase 3)
VERSION	3 — 12 September 2026
GOVERNED BY	Charter v3 (Phase 1) • SRS v3 (Phase 2)
CONTAINS	21 design decisions • 9 corrections • 2 named deliverables • TPC-DS structure and query style verified against source

What this document is for. The charter fixed why the project exists; the SRS fixed what the system must do. This document fixes *how* — the parts, their boundaries, the algorithms inside them, the shape of the data that moves between them, and the order in which they get built. Every decision here records the option it was chosen against. Section 13 lists eight corrections — six from a whole-design coherence pass, two from a verification pass against the TPC-DS specification that proved a recommendation of mine unsound.

01 How to read this document

Design decisions are numbered permanently and each carries the alternative it beat.

Decision identifiers

DD-01 onward, permanent, never reused. They join the charter's **D-01** to **D-08** (project-level decisions) and are referenced from code comments and commit messages. Requirement identifiers — **FR**, **NFR**, **DR**, **IR** — refer to the SRS. Threat identifiers **T-01** to **T-08** refer to SRS §09; risk identifiers **R-01** to **R-09** refer to Charter §08.

The one principle everything below follows

Cartograph is a **pure core wrapped in a thin shell of things that talk to the world**. Section 02 establishes this and every later section obeys it. If a component in this document appears to hold both computation and I/O, that is an error in the document, not a design choice.

What is deliberately not here

Test suite structure — fixtures, what runs without a database, how the negative cases are organised. NFR-26 fixes *what* must be tested; *how* the suite is arranged is build-phase work and designing it in the abstract now would be guessing.

02 Architecture

Most of what makes Cartograph interesting is pure computation. That fact organises the whole system.

DD-01 • The organising principle: pure core, impure edges

Sort every responsibility in the system by whether it must touch the outside world.

Touches the world: reading the warehouse catalog, generating embeddings, calling the model API, executing SQL, reading and writing the application store.

Pure computation: building the graph from schema facts, finding paths through it, selecting one, rendering a path into English, assembling the prompt, parsing and validating SQL, checking path conformance.

Note what falls in the second list. Path finding, validation, conformance, explanation — **the components that make Cartograph what it is are all pure**. That is not a coincidence to be noted and forgotten; it is a fact to build on.

Three consequences, each of which matters specifically to this project:

- **Testable without infrastructure.** A pure component needs no database, no Docker, no API key. NFR-26 demands negative tests on the validation and path-finding layers — SQL that must be rejected, SQL that joins wrongly while appearing to conform. Those tests run in milliseconds against fixtures. Under a design where validation held a database connection, each of them would need a live warehouse.
- **Readable alone.** The owner's stated method is to build with assistance and understand afterwards. A pure component can be read and understood in isolation, because everything affecting its output is visible in its arguments. A component entangled with I/O can only be understood by understanding everything it touches.
- **Cannot half-fail.** A pure function either returns or raises; it cannot leave a connection open, a row half-written or a cache poisoned. This removes an entire class of bug from the parts of the system that matter most.

REJECTED ALTERNATIVE

A conventional service-layer architecture, where each service owns its own database access — the shape most tutorials teach. It is simpler to write initially and much harder to test, and it would have put a live warehouse connection inside the component this project most needs to prove correct.

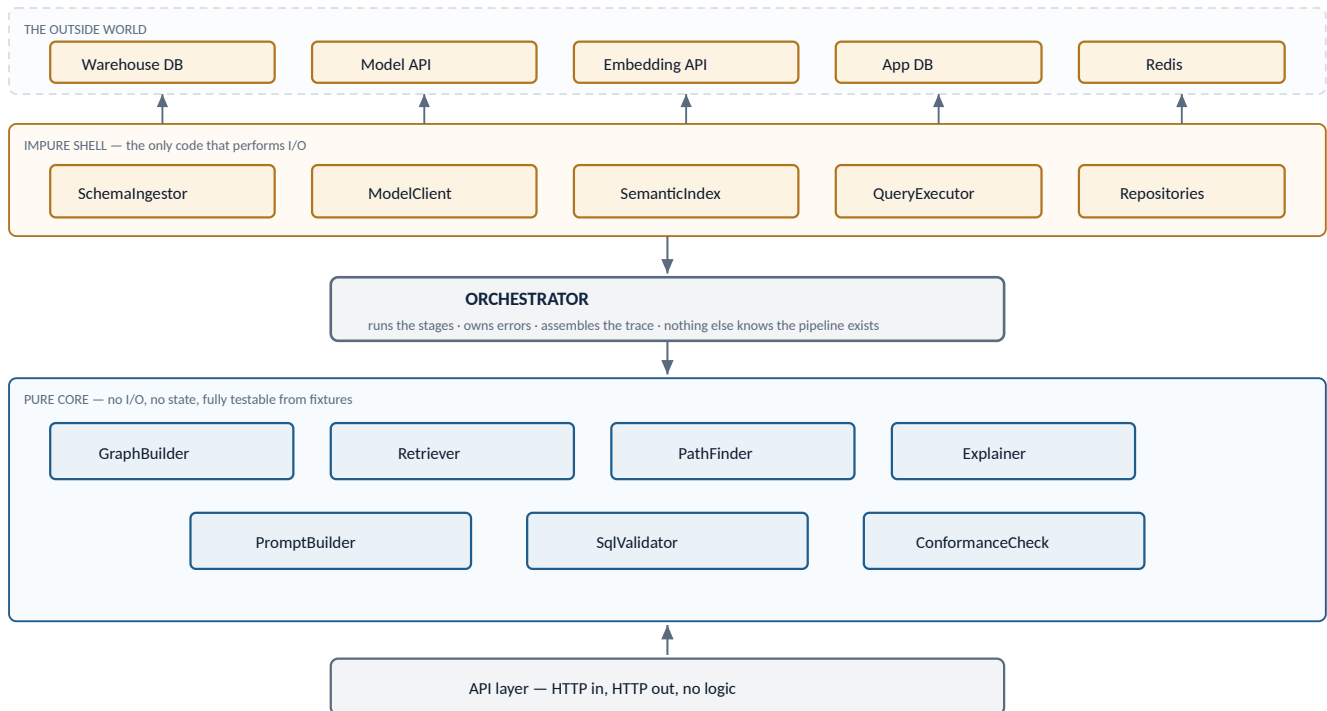


Figure 1 — Layered architecture. Amber components perform I/O and are the only ones that may. Blue components are pure functions over data. The orchestrator sits between them and is the sole component aware that a pipeline exists; the stages never call one another.

Component catalogue

COMPONENT	NATURE	RESPONSIBILITY — AND WHAT IT EXPLICITLY DOES NOT DO
API layer	impure	Translate an HTTP request into a call and the result back into HTTP. <i>Holds no logic</i> . If a decision is being made here, it belongs elsewhere.
Orchestrator	impure	Run the stages in order, collect what each returns, assemble the trace, decide what happens on failure. <i>The only component that knows the pipeline exists</i> .
Schemalngestor	impure	Connect to the warehouse, read the catalog, apply the overlay, build readable descriptions. Produces a flat snapshot of schema facts. <i>Does not build the graph</i> .
GraphBuilder	pure	Snapshot → graph. Nodes, edges, edge provenance. <i>Never touches a database</i> — which is why it can be tested against a fixture snapshot in milliseconds.
SemanticIndex	impure	Embed schema descriptions; answer similarity queries. Wraps both the embedding API and pgvector.
Retriever	pure	Question + scored matches → bounded anchor set, with per-candidate sub-scores and column-level alternatives. <i>Receives similarity results; does not fetch them</i> .

COMPONENT	NATURE	RESPONSIBILITY — AND WHAT IT EXPLICITLY DOES NOT DO
PathFinder	pure	Anchors + graph → all discovered paths, the selected path, the rule applied, and the bounded subgraph. Owns <i>the second bound</i> (DD-11).
Explainer	pure	Path + readable names → one plain-English sentence per path.
PromptBuilder	pure	Subgraph + selected path + question → the exact text sent to the model.
ModelClient	impure	Prompt → text, tokens, cost. The adapter of IR-13; the only place a provider name appears.
SqlValidator	pure	Parse; reject non-SELECT; reject unknown references. <i>Never executes anything.</i>
ConformanceCheck	pure	Extract the joins a SQL statement actually performs and compare them to a given path. Used twice: once in validation, once as the evaluation instrument (DD-12).
QueryExecutor	impure	Validated SQL → rows, on the read-only connection, under timeout.
Repositories	impure	One per stored thing. The only components that touch the application database.

DD-02 • Three boundary rules

Rule 1 — the trace is assembled by the orchestrator; stages never write to it. Each stage *returns* its contribution and the orchestrator collects. This makes NFR-13 — a trace is never lost — structural rather than hopeful: there is no partial write to lose, because nothing is written until the orchestrator writes it once.

Rule 2 — two database connections that never mix. The warehouse connection uses a role granted SELECT only (NFR-07). The application connection is ordinary read-write. Different credentials, different configuration keys, separate connection objects. **No component holds both.** This is what makes NFR-07 impossible to violate by accident rather than merely forbidden.

Rule 3 — the frontend reaches the backend through exactly one module (IR-16). No exceptions, from the first request onwards.

DD-03 • Deployment shape

One Postgres instance, two databases, two roles. Rejected: two separate instances. One instance is simpler to run, one fewer container, and NFR-07's guarantee comes from the *role* rather than from physical separation, so separation buys nothing it doesn't already have. Accepted cost: an instance failure takes both databases, which for a demonstration system is tolerable.

DD-04 • Error ownership

The orchestrator owns error handling. Stages return outcomes — including failure outcomes — rather than raising their way out of the pipeline. Rejected: per-stage exception handling. Returning outcomes keeps the stages pure, keeps every error path in one readable place, and means **a failure still produces a trace** rather than losing one.

03 Request lifecycle

One question, seven stages, each contributing a named part of the trace.

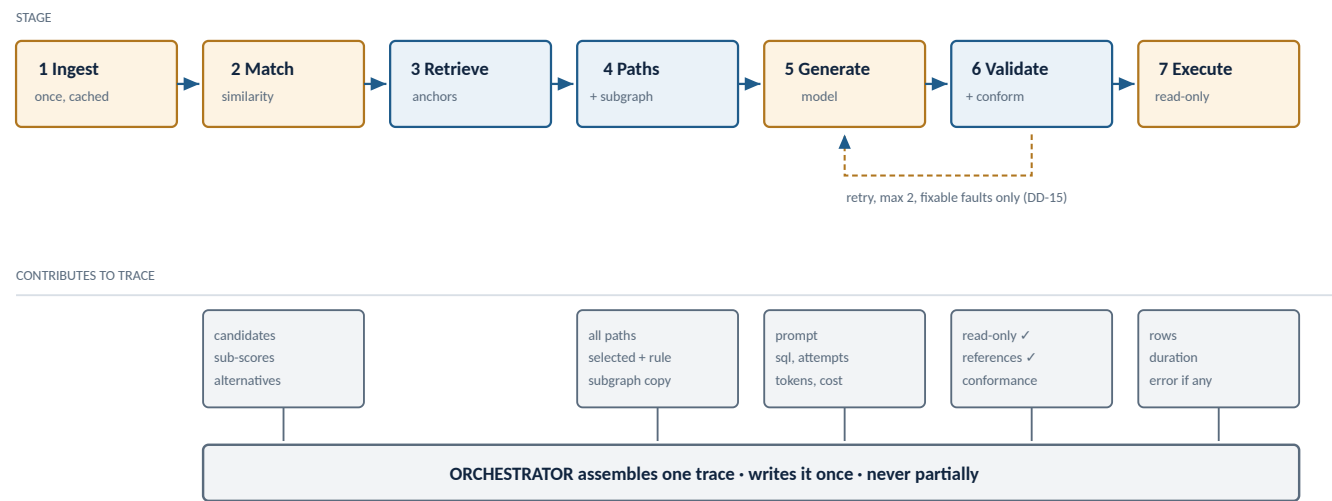


Figure 2 — Request lifecycle and trace assembly. Stage 1 runs once at ingestion, not per question. Every stage returns its contribution upward; the orchestrator writes the assembled trace exactly once. A failure at any stage still produces a trace containing everything the stages before it returned.

Timing. Each stage emits a structured log entry recording its duration in milliseconds (NFR-22). Those durations are the evidence for NFR-01 through NFR-05, and they are also carried in the trace's timings field so they are visible per query rather than only in aggregate.

Synchronous by design (D-05). One request, one response, connection held for the duration. The stage indicator in the interface (IR-12) is driven client-side from the known stage ordering, not from server events — which is what keeps this simple. Accepted cost: a query that outruns the HTTP connection loses its answer even though it ran and its trace was written.

04 The trace object

One structure doing four jobs: what is stored (DR-09), what crosses the API (IR-04), what the panel renders, and what

criteria 2, 13 and 14 are checked against.

Trace			
└ trace_version	int, present from the first commit		
└ query_id · user_id · question · created_at			
└ outcome	answered not_answerable validation_failed model_failed execution_failed		
└ schema_ref	snapshot id + hash of the schema this ran against		
└ retrieval			
└ candidates[]	kind (table column) · name · parent table · semantic_score · keyword_score · combined · rank		
└ anchor_bound	limit applied, and what it excluded		
└ term_alternatives[]	term · columns considered · column chosen	FR-41	
└ subgraph	nodes and edges handed to the model – a copy, not a reference	DD-05	
└ paths			
└ discovered[]	└ Path: id · edges[] (from/to table+column, source: catalog overlay) · length · description (plain English)	FR-40	
└ selected_id	the path the system chose		
└ rule	which rule chose it, incl. "tie broken alphabetically"		
└ actual_edges[]	the joins the generated SQL really performed	DD-13	
└ diverged	true when actual ≠ selected	DD-13	
└ subgraph_bound	limit applied at path level, and what it excluded		
└ generation			
└ attempts[]	prompt · model id · sql · tokens in/out · cost · outcome of this attempt	DD-15	
└ validation			
└ read_only	pass fail		
└ references	pass fail		
└ conformance	conforms diverged incomplete not_checked	DD-13	
└ findings[]	what specifically failed, and where		
└ execution	row count · duration · error if any		
└ timings	milliseconds per stage	NFR-22	
└ narrative	the stored plain-English account	FR-26	

Optionality. Every section except question, outcome and timings is optional. A question that fails validation yields a trace with retrieval, subgraph, paths, generation and validation populated and execution empty. A question that cannot be answered at all yields retrieval and an outcome. **There is no path through the system that produces no trace**, and the panel renders whatever is present — so a failure is as legible as a success.

DD-05 • The trace carries its own copy of the graph

Rejected: storing only table and column names and re-rendering against the live schema at view time.

FR-05 permits the schema to be re-ingested and replaced. Under the rejected option, every historical trace would then re-render against the *new* schema — a renamed column, an added overlay edge, a dropped table — and the picture would silently change.

The trace would show something that did not happen, with no error and no indication. That is the precise failure this project exists to prevent, occurring inside the mechanism built to prevent it.

FR-10 and DD-11 bound the subgraph, so the copy is small — a few kilobytes. Accepted cost: the same subgraph is duplicated across every trace that touched it. In a storage-constrained system that would argue for the alternative; here it does not.

DD-06 • The narrative is composed once and stored

Rejected: composing the sentence at read time from the stored facts.

The decisive argument is not philosophical but structural. Composing at read time places the logic either in the frontend — where database semantics do not belong and cannot be cleanly tested — or in every backend path that returns a trace, including history, where it must then handle every historical trace shape forever. Storing it keeps the Explainer a small pure component that runs once, and keeps the frontend a renderer of strings.

Accepted cost, stated plainly: improving the wording later does not improve traces already written.

DD-07 • Storage — document body plus extracted columns

Rejected: full normalisation into five linked tables; and pure JSONB with no extracted columns.

A trace is written once, read whole, never updated and never joined against. That is a document, not a relation. But five questions *will* be asked across traces — how often ambiguity appeared, how often conformance failed, what each query cost, what the outcome was, how long it took — and asking those of a JSON body is awkward and error-prone.

So: the body lives in a JSONB column; outcome, had_ambiguity, conformance_result, cost_estimate and duration_ms are lifted into ordinary columns at write time.

THE OBJECTION, AND ITS ANSWER

This is a project about databases, and its most interesting structure is stored as a blob. Correct, and deliberate. The trace is the **one** document in an otherwise fully relational schema — users, queries, snapshots, elements, edges, evaluation questions and runs are all normalised with real foreign keys. Structural enforcement for the trace lives at the application boundary instead (IR-11, Pydantic), which is where FastAPI services conventionally put it. A malformed trace cannot be written because a malformed trace cannot be constructed.

Versioning. trace_version exists from the first line of code. The shape will change during the build; a version integer costs nothing now and means a trace written at step 4 still renders at step 14.

One field that grows without bound. generation.attempts[].prompt stores full prompt text — a few kilobytes each, one to three per query. Required by FR-17, and the system cannot defend its own behaviour without it. Noted because if storage ever becomes a problem, this is where it came from.

05 Algorithms

Where the project's real engineering sits. Eight decisions, presented in pipeline order.

DD-08 • Embed enriched descriptions, not raw identifiers

Rejected: embedding column and table names as they appear in the catalog.

Semantic retrieval compares the question to the schema in embedding space. TPC-DS names are compressed identifiers:

```
ss_ext_sales_price    i_category    ca_state    d_moy    ws_net_paid
```

A user asks "which product category made the most revenue in the western region?". The word **revenue** must reach `ss_ext_sales_price`. In embedding space it does not — the model sees a token fragment, not English. `d_moy` means "date: month of year" and reads as nothing at all. This is threat T-03, and it lands directly on the component the project stands or falls by.

The design: at ingestion time, each column gets a readable description built from three sources, in priority order —

- the column's database comment, where one exists (a real description always beats a derived one);
- otherwise, prefix and abbreviation expansion from the overlay's naming section — `ss_` → store sales, `ca_` → customer address, `ext` → extended, `moy` → month of year;
- the parent table's readable name and the column's data type, appended as context.

```
ss_ext_sales_price → "store sales – extended sales price.  
Numeric column in table store_sales."
```

That text sits far closer to "revenue" than the identifier ever will. Where no mapping applies, the raw name is used — so an incomplete mapping degrades gracefully to the rejected option rather than breaking.

WHY THIS EFFORT IS JUSTIFIED WHEN EFFORT IS REFUSED ELSEWHERE

This is the one place where the warehouse choice actively works against the project. TPC-DS was selected for size and credibility (Charter D-01) and paid for in name quality; Pagila's names read like English and would not need this. This is that bill arriving. Skipping it risks the opening question of the demonstration video failing to locate the right column.

DD-09 • Hybrid scoring — semantic plus literal

Rejected: embedding similarity alone.

Watch two words from the same question behave differently.

TERM	TARGET	BEHAVIOUR
"category"	<code>i_category</code>	Appears <i>literally</i> . Keyword search finds it with certainty. Embeddings probably find it too — but <code>i_class</code> and <code>i_manufact</code> are also product attributes sitting nearby in meaning-space, so a similar-but-wrong column can outrank the exact match.
"revenue"	<code>ss_ext_sales_price</code>	Appears literally <i>nowhere</i> . Keyword search returns nothing. Only the embedding of the enriched description connects them.

The two methods fail in different places and neither covers the other's blind spot — the textbook case for combining rather than choosing. Postgres provides full-text search natively, so this is a second index and a second query, not a new dependency.

Combination: normalise both scores to 0–1, then $\text{combined} = \alpha \cdot \text{semantic} + (1 - \alpha) \cdot \text{keyword}$, with α starting at 0.5 and tuned against the frozen evaluation set (DR-16).

The bug to avoid, stated now because it produces plausible output. Cosine similarity occupies a tidy 0–1; the text-rank score does not, and its range varies with the corpus. **Normalise before combining or the weighting is meaningless** and one signal silently dominates while α appears to be doing something. Nothing errors, results look reasonable, and tuning α has no effect — which is exactly the class of fault hardest to notice.

A property gained for free. Hybrid scoring yields two sub-scores per candidate, both recorded in the trace. The trace can therefore say *this column was retrieved because your wording matched it literally* versus *this one was retrieved on meaning, not wording*. That is explainability obtained as a side effect, in a project whose thesis is explainability. A single opaque similarity number offers nothing to say.

DD-10 • Expand along the graph; scores locate, structure completes

Rejected: passing the top- k scoring tables directly to the model.

A ranked list has a problem document retrieval never has: **the results must connect**. You cannot write SQL over three tables that do not join.

Worse, some tables are structurally necessary and score badly. `date_dim` carries "last quarter" but a table of date attributes resembles no particular phrasing strongly. And an *intermediate* table connecting two anchors may score near zero — it is mentioned nowhere in the question; it is simply in the way.

The design: top-scoring tables become **anchors**. The Pathfinder then walks foreign-key edges to find routes between anchors and pulls in every table lying on such a route, regardless of its score.

Two guards:

- **Hop limit.** Routes longer than four edges are not considered. Without this, distant anchors are connected through half the schema by paths that are structurally valid and semantically meaningless.
- **Disconnection is reported, not repaired.** If no route within the hop limit connects two anchors, that is recorded as a finding. **A disconnected anchor set is information** — frequently it means the question cannot be answered from this schema, which is FR-42 — and forcing a connection to make the output look tidy would be a fabrication.

WHY THIS DECISION IS THE PROJECT

Without expansion, Cartograph is vector search with a graph drawn beside it for decoration — the structure never does any work. With it, **score tells the system where to look and the graph tells it what must come along**. That distinction is what the project's name claims and what an examiner will probe. It also yields FR-12 at no extra cost: walking between two anchors discovers *every* route between them, which is precisely the ambiguity detection the thesis rests on. Expansion and path-finding are one operation seen from two angles.

A concrete instance, verified 12 September. In TPC-DS query q3, `date_dim` and `item` are never joined to each other and *cannot* be — they share no column. They connect only through `store_sales`. So a question about sales by brand in November names neither the fact table nor the route, yet answering it requires both. Score alone would never retrieve `store_sales`; the graph has to supply it.

DD-11 • Two bounds, two owners, and a protected set

Rejected: a single cap applied by score; and per-table column pruning.

There are two distinct limits, and conflating them was an error corrected during the coherence pass (§13, correction 5).

BOUND	OWNER	RULE
Anchor bound	Retriever	How many scored tables are proposed as anchors before expansion begins.
Subgraph bound	PathFinder	How large the expanded subgraph may become. Default ten tables, tuned against the evaluation set.

The protected set. A blind cap can remove a connector table and leave anchors that no longer join — the model then receives the right tables, the right measures, and no way to relate them. So:

- Anchors and **every table lying on a discovered path between anchors** are protected from the cap.
- If the protected set alone exceeds the cap, the fix is to drop the **lowest-scoring anchor** and re-expand — never to cut a connector.
- Whatever survives, **primary key and foreign key columns are never removed**. They score terribly — `ss_item_sk` describes nothing a human would ask about — so under any score-based pruning they are among the first casualties, and their loss is silent and fatal.

Columns are not pruned within an included table. `store_sales` holds `ss_ext_sales_price`, `ss_net_paid`, `ss_net_profit` and `ss_sales_price` — several plausible readings of "revenue" with similar scores. Showing the model two of them and hiding the rest does not help it choose; it *biases* the choice, and the trace cannot reveal why because the hidden column is simply absent. **Giving the model a whole table is honest; giving it a curated slice is a hidden editorial decision.**

Both bounds record what they excluded in the trace. If an answer was wrong because the eleventh table was cut, the trace says so. A bound that discards silently reproduces the project's own target failure one level down.

DD-12 • Path selection: shortest, then deterministic arbitrary, with optional declared preference

Rejected: cardinality or domain heuristics; refusing to answer when tied.

Shortest path resolves most cases. It does not resolve the case the project exists for.

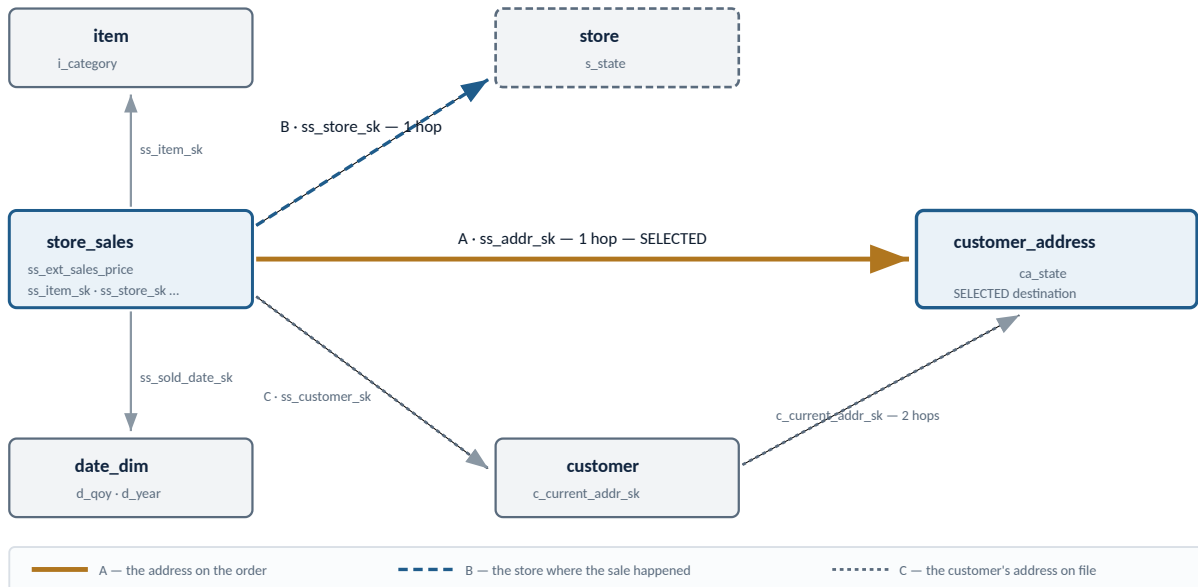


Figure 3 — The demonstration case: three routes from a sale to a region. A and B are both one hop, so the shortest-path rule cannot separate them. C reaches the same table by a different meaning. **Verified 7 September against the TPC-DS schema and the toolkit's published constraint file:** `ss_addr_sk → customer_address(ca_address_sk)`, `ss_store_sk → store(s_store_sk)`, `ss_customer_sk → customer(c_customer_sk)`, and `customer.c_current_addr_sk` all exist, as do `ca_state` and `s_state`. All three routes are real.

The rule, in three parts:

1. **Shortest path wins.** Fewest edges.
2. **Ties break deterministically and arbitrarily** — alphabetically by the sequence of table names. Alphabetical rather than by declaration order because declaration order can change between ingestions and alphabetical cannot; the same schema always yields the same choice.
3. **An operator may declare a preference** in the overlay's preferences section. Absent one, rule 2 applies and the trace says the choice was arbitrary.

WHY THE ARBITRARINESS IS DELIBERATE

A heuristic here would not be a general rule — it would be an assumption about a *business*. For a bricks-and-mortar chain the store path is right; for an online retailer the delivery address is right. **Cartograph reads a schema and has no business context whatsoever**, so any preference it invented would be a silent assumption compiled into a tool that claims not to make them. And a system that appears to choose cleverly invites the user to stop checking — which is the exact behaviour the project exists to discourage. The declared preference in the overlay is the correct place for that knowledge: supplied by someone who has it, recorded where it can be seen.

DD-21 • The warning fires when the choice was arbitrary, not when alternatives existed

The verification pass changed the shape of this problem. The graph carries roughly **eighty** edges, not thirty, and `catalog_sales` alone holds both `cs_bill_addr_sk` and `cs_ship_addr_sk` — billing address against shipping address, another genuine ambiguity. **Multiple valid paths will be the common case, not the exception.** A warning that fires on every question is not a warning.

The distinguishing fact is already computed. If one route is one hop and another is three, the shortest-path rule made a *principled* choice — it had a reason. If two routes are **tied**, the rule had nothing, and rule 2 above fell through to alphabetical. That is the arbitrary case.

SITUATION	PANEL	WHAT IT SAYS
Tied paths, no declared preference	Loud	Amber block at the top of the panel — in substance, <i>"I had no basis for this choice."</i>
Alternatives existed, shortest won	Quiet	One line: <i>"Three routes existed; the shortest was used."</i>
Resolved by declared preference	Quiet	One line, naming the preference and where it was declared.

Nothing is hidden in any tier — every discovered path stays in the trace and stays drawn. What changes is what interrupts the reader. The alarm now carries a precise meaning, and it is a sharper claim than the one it replaces: the system is not warning about complexity, it is warning that **its own selection had no reason behind it**.

An observation from the worked example. Alphabetically, `customer_address` precedes `store`. So on the headline question the system selects the *address on the order* — while most people's intuition says a sale's region is the store's region. The system therefore picks the less obvious reading and announces that it did. That is a stronger demonstration than a correct-looking guess: it produces the moment where a viewer asks *"wait, is that what I meant?"*, which is the entire point.

DD-13 • Conformance: three outcomes, flag rather than reject

Rejected: exact set equality; rejecting divergent SQL without executing.

The comparison is between the join conditions `sqlglot` extracts from the generated SQL and the edges of the selected path. Exact equality produces constant false alarms — the model may legitimately use fewer tables than offered — while checking nothing meaningful is threat T-02 realised.

Amended 12 September, on evidence from the warehouse itself. Version 2 scoped the v1 checker to top-level JOIN . . . ON nodes with alias resolution, returning `not_checked` for everything else. Reading TPC-DS query q3 during Day Zero showed that **the benchmark's own queries contain no JOIN keyword at all** — tables are comma-separated in FROM and the join predicates sit in the WHERE clause as column-to-column equalities.

That form is therefore not exotic; it is the house style of the schema the model is being shown. A checker blind to it would pass every hand-written test, then abstain on a large share of real queries — **silently shrinking the evaluation denominator while appearing to work**. Abstention that frequent is indistinguishable from failure.

So v1 extracts both forms: the ON clause of explicit joins, *and* equality predicates in WHERE between two qualified columns. Mechanically these are the same thing — an equality between two column references — so the cost is small and the coverage gain is large.

The formulation:

- every edge of the selected path appears among the SQL's join conditions — *whether written as an ON clause or as a WHERE equality between two qualified columns*; and
- no join condition in the SQL uses an edge outside the selected path.

OUTCOME	MEANING AND CONSEQUENCE
conforms	Path edges all present, nothing foreign. Proceed.
diverged	The SQL joined along edges outside the selected path. Execute anyway ; record <code>paths.actual_edges</code> , set <code>paths.diverged</code> , and render <i>the actual path</i> in the graph alongside the selected one.

OUTCOME	MEANING AND CONSEQUENCE
incomplete	Some path edges unused — the model needed fewer tables. Usually benign. Proceed, recorded.
not_checked	The SQL contains a construct the checker cannot analyse with confidence. Recorded with the reason.

Flag rather than reject, and why. The lie this project prevents is not "the model chose a different path" — it is "the trace claims one thing while the SQL did another." Rejecting avoids the lie by refusing to speak, and discards a possibly-correct answer along with the information that the model disagreed with the selection rule. Flagging tells the truth: the user gets the answer, sees the route actually taken, and sees that it was not the one chosen.

not_checked is the whole antidote to T-02. Where the checker meets aliases it cannot resolve, a join inside a nested subquery or CTE, a self-join, or an unfamiliar construct, it records not_checked with a reason — **never conforms**. Note what is *not* on that list any more: implicit WHERE-clause joins are handled, not abstained on. A checker that quietly degrades to "looks fine" is worse than no checker, because it manufactures confidence in precisely the place the project promises not to. NFR-26 requires negative tests for exactly this: SQL written deliberately to join differently while appearing to conform. If those tests pass the checker, the checker is decorative.

DD-14 • Prompt construction

The subgraph is rendered as CREATE TABLE declarations with readable descriptions as comments, followed by the selected path written out as explicit join conditions. **Rejected:** prose and JSON renderings — models handle DDL better, having seen schemas in that form far more often.

The model returns JSON containing a single field: the SQL. **Rejected:** free text, which requires extracting SQL from prose and code fences.

The model does not explain itself. Every explanation in the trace is derived by Cartograph from facts it holds — the paths found, the rule applied, the edges supplied. A model-written explanation is unverifiable narration: it can assert anything, and nothing in the system could check it. In a project whose thesis is that reasoning must be inspectable, an unverifiable explanation is worse than none.

DD-15 • Retry policy

Rejected: no retry; and unconditional retry.

On a validation failure the error is fed back to the model and generation is attempted again, capped at two retries (NFR-15). Every attempt — including rejected ones — is recorded in `generation.attempts[]`.

Retry only fixable faults: a non-existent column, a syntax error, a path divergence. **Never retry a rejected write operation.** If the model produced DROP, DELETE or UPDATE, that is not a slip to be coached out of — it is a signal, it is recorded prominently, and the query ends there. Retrying it would mean nudging a model toward passing a safety check, which inverts the purpose of the check.

A SUBTLETY WORTH KNOWING BEFORE IT HAPPENS

Telling the model "use this path instead" after a divergence will produce compliance. Compliance is not correctness: the model may have diverged because the shortest-path rule chose a worse route, and the retry then argues it out of a better answer into a conforming one. The conformance rate improves while the SQL gets worse. Watch for repeated divergence of the same shape — it is evidence about the selection rule, not about the model.

DD-16 • The overlay file

One YAML file per warehouse, three sections, loaded at ingestion. **Rejected:** three separate files — all three are the same kind of thing, facts about this warehouse that the catalog cannot supply.

For TPC-DS the relationships section is generated, not written. The toolkit publishes `tpcds_ri.sql` — roughly eighty ALTER TABLE ADD CONSTRAINT statements authored by the benchmark's own authors. Parsing that file into the overlay takes minutes and removes the possibility of a mis-transcribed edge entirely. The load script *also* attempts to apply the statements as real constraints — after adding primary keys to the seventeen dimension tables, since a foreign key needs a unique target — so that on a successful apply the catalog itself declares them and the project's founding claim holds literally.

Merge rule: catalog edges win; the overlay fills what the catalog does not declare; provenance is recorded on every edge either way.

```
relationships:  # foreign keys the catalog does not declare      FR-43
  - from: store_sales.ss_store_sk
    to:   store.s_store_sk

naming:        # prefix and abbreviation expansions          DD-08
  prefixes: { ss_: store sales, ca_: customer address, d_: date }
  words:    { ext: extended, moy: month of year, sk: key }

preferences:   # optional declared resolution of a known ambiguity DD-12
  - between: [store_sales, customer_address]
    prefer:  via_store
```

Every edge originating here is marked `source: overlay` in the graph and shown as such in the trace, so a reader can always distinguish what the database declared from what a human asserted.

06 Persistence

Eight tables, fully relational but for one deliberate document.

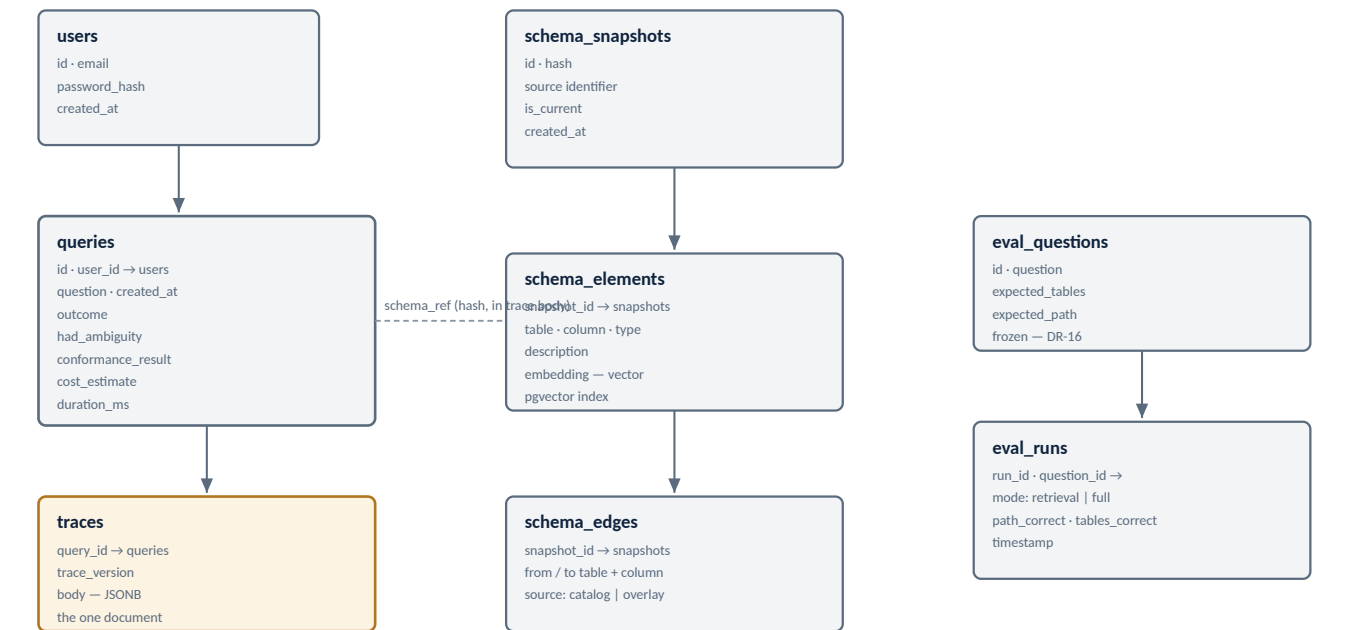


Figure 4 — Application store. Seven normalised tables with real foreign keys, and one JSONB document. Redis holds the answer cache and rate-limit counters and appears nowhere here, being neither durable nor relational.

DD-17 · Four choices inside that layout

CHOICE	REASON
Traces are a separate table, not a column on queries	History listing reads many rows and never needs the bodies. Separation makes it impossible to accidentally drag megabytes of JSON into a list view.
The graph is stored as edge rows, not as JSON	Keeps the rule of DD-07 honest — the trace is the <i>one</i> document. The in-memory NetworkX graph is constructed from these rows at load.
Snapshots are added, never replaced	FR-05 marks a new snapshot current rather than deleting the old. They are small, and a trace's schema_ref then never points at something that no longer exists.
Schema elements are normalised rows	Forced: pgvector requires a real column to index. No alternative existed.

User identity from the first migration. queries.user_id exists from the moment the table does — not added later when authentication arrives at build step 11. Retrofitting identity into a populated table is the expensive version of this; declaring it up front costs nothing (DR-08).

All schema changes flow through Alembic migrations (DR-14). No manual edits, at any point, including during the build.

07 Frontend

Five pages, one API client, and a panel designed against the reader rather than the data.

Pages: /ask · /history · /history/[id] · /login · /register. Nothing else.

Components: QuestionInput · ResultTable · TracePanel · GraphView · StageIndicator · ExampleQuestions. All client-side; this is an interactive application, not a content site, so server rendering buys nothing.

One API client module (IR-16). Every request without exception, from the first one. This is what makes authentication at step 11 a single edit rather than a sweep through every call site — and it costs nothing if done first.

DD-18 · What the graph draws

Tables are nodes; foreign keys are edges; **columns are listed inside a table node rather than being nodes themselves**. The internal graph model treats columns as nodes, but rendering them that way produces the hairball NFR-18 forbids. The chosen path is drawn solid and prominent, alternatives dashed and dimmer, and hovering an edge reveals its plain-English description. Where the SQL diverged (DD-13), the *actual* path is drawn too, distinctly, alongside the selected one.

DD-19 · The trace panel leads with plain English, not with data

Rejected: a flat panel presenting everything in pipeline order.

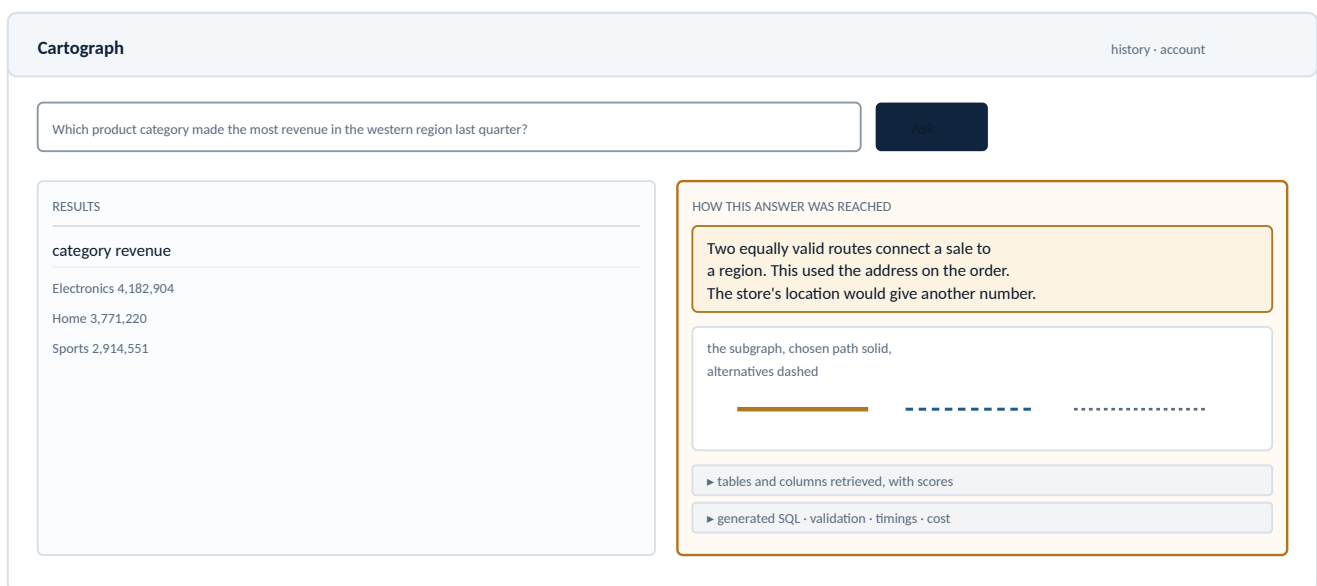


Figure 5 — The ask screen. The ambiguity statement occupies the top of the trace panel in plain language. The graph sits below it. Everything technical — scores, SQL, validation detail, timings, cost — is collapsed behind two disclosure rows. On an unambiguous question the amber block is absent and the panel is quiet.

Why the flat alternative fails despite being more honest-looking. The engineer's instinct is to lead with the SQL — it is the concrete artefact and everything else is commentary. But the SQL is the *least* useful item for the person the panel is written for. A manager scanning a wall of technical output concludes it is not addressed to them and stops reading, and the ambiguity warning — the sole reason the panel exists — goes unread three scrolls down. **A trace that is complete but unread has failed.** Criterion 14 is worded as it is precisely to prevent that outcome.

Prominence proportional to ambiguity. A clean question with one obvious path produces a quiet panel with details collapsed. A question where two equally valid paths existed produces a statement at the top, in a form the reader cannot skim past. The failure this project is built around is a confident number nobody thought to question; burying the warning politely would recreate that failure behind a better data structure.

08 One question, end to end

Every decision in this document, applied to a single question, in order.

"Which product category generated the highest revenue in the western region last quarter?"

#	STAGE	WHAT HAPPENS, CONCRETELY
0	Already done at ingestion	24 tables read from the catalog. Foreign keys added from the overlay's <code>relationships</code> section, since TPC-DS declares none. Descriptions built: <code>ss_ext_sales_price</code> → "store sales — extended sales price"; <code>d_qoy</code> → "date — quarter of year"; <code>ca_state</code> → "customer address — state". All embedded and stored with the snapshot.
1	Match SemanticIndex	The question is embedded and run against the index; in parallel a full-text search runs over the same descriptions. Two score sets return.
2	Retrieve Retriever	"category" — literal keyword hit on <code>i_category</code> , high certainty. "revenue" — no literal match anywhere; semantic hit on <code>ss_ext_sales_price</code> via its description, with <code>ss_net_paid</code> close behind, recorded as a term alternative. "region" — reaches both <code>s_state</code> and <code>ca_state</code> . "last quarter" — reaches <code>d_qoy</code> . Anchors: <code>store_sales</code> · <code>item</code> · <code>store</code> · <code>customer_address</code> · <code>date_dim</code> .
3	Expand and bound PathFinder	Foreign keys are walked out of <code>store_sales</code> . Item: one hop. Date: one hop. Store: one hop. Customer address: one hop <i>and again</i> at two hops via customer. No additional connector tables are required — every anchor is directly reachable — so the protected set is five tables and the ten-table bound is not reached. Nothing is excluded; the bound records that.
4	Select PathFinder	Region is reachable two ways at one hop. The shortest-path rule cannot separate them. No preference is declared in the overlay. The tie breaks alphabetically: <code>customer_address</code> before <code>store</code> . Selected path: <code>store_sales</code> → <code>customer_address</code> . Rule recorded as "tied at one hop; broken alphabetically; arbitrary."
5	Explain Explainer	Three sentences composed and stored: <i>the address on the order</i> · <i>the store where the sale happened</i> · <i>the customer's address on file</i> .
6	Prompt PromptBuilder	Five tables rendered as <code>CREATE TABLE</code> with descriptions as comments, all columns present including keys, followed by the selected join condition written explicitly. Roughly a few thousand tokens.
7	Generate ModelClient	One call, temperature 0. JSON back, containing SQL. Tokens and cost recorded on the attempt.
8	Validate SqlValidator + Conformance	Parsed. Read-only: pass. References: every table and column exists in the snapshot. Conformance: the joins extracted from the SQL are compared to the selected path — conforms, or diverged with <code>actual_edges</code> recorded, or <code>not_checked</code> if a construct defeats the extractor.
9	Execute QueryExecutor	Run on the SELECT-only connection under a timeout enforced at the connection, not in application code. Rows returned.
10	Assemble Orchestrator	Everything returned by stages 2–9 is collected into one trace and written once. Extracted columns populated: <code>had_ambiguity</code> = <code>true</code> , <code>conformance_result</code> , <code>cost</code> , <code>duration</code> .

#	STAGE	WHAT HAPPENS, CONCRETELY
11	Render TracePanel	The panel opens with the ambiguity statement. Below it, the subgraph with the selected path solid and the two alternatives dashed. Below that, the results. Everything technical collapsed.

THE TWO PLACES REALITY MAY DISAGREE

Whether date_dim scores highly enough to be retrieved at all, and whether the foreign-key structure is as recalled. **Both are answered on the first day, at build step 2**, before anything is built on top of them. Everything else above follows from decisions in this document.

09 Ordered build sequence

Named deliverable. Fourteen steps, each one a checkpoint.

Five ordering principles. Risk first — anything that could invalidate everything else goes before anything is built on it. Dependency-free first — pure components before ones needing an API key. Ground truth before tuning. The centrepiece does not go last. Core before Supporting.

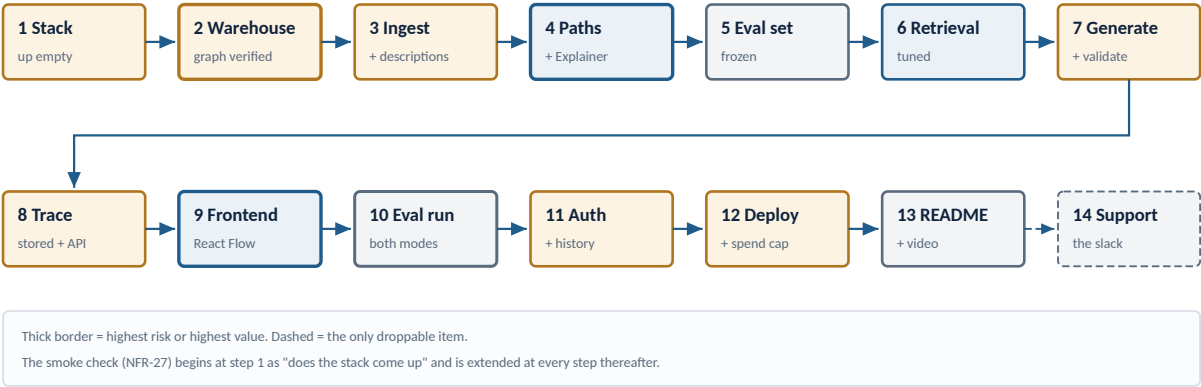


Figure 6 — Build order. Step 2 answers threat T-01 before anything rests on it. Step 4 delivers the project's central component with no external dependency. Step 5 freezes ground truth before step 6 tunes against it. Step 14 is the only item whose loss costs no definition-of-done criterion.

#	DELIVERABLE	WHY IT SITS HERE
1	Stack stands up empty. WSL2, Docker, Postgres + pgvector, Redis, FastAPI and Next.js hello-worlds, one docker compose up. Repo public from the first commit; CI running one trivial test.	R-03. A stack that cannot stand up empty will not stand up full. Gets NFR-20 and NFR-21 satisfied now rather than as a scramble at the end.
2	Warehouse loaded, graph verified. TPC-DS generated via DuckDB, loaded, foreign keys added from the specification — <i>and a test asserting the expected edge</i>	T-01 and R-09, the highest-risk item in the project, answerable on day one. Nothing downstream can be trusted until this passes. The storage check answers T-05 before deployment rather than during it.

#	DELIVERABLE	WHY IT SITS HERE
	<i>count and the expected multi-path pairs. Plus the five-minute free-tier storage check.</i>	
3	Schema ingestion, descriptions, graph in the application. FR-01, 02, 03, 43 and DD-08.	Descriptions belong here, not with retrieval — step 4's Explainer needs readable names, and without them it would emit raw identifiers (§13, correction 3).
4	Path finding, selection, explanation. FR-11–14, FR-40. Plus a script that prints every path between two named tables.	Needs nothing external — no API key, no embeddings, no network. The project's central component, built and thoroughly tested on day two, with tests running in milliseconds. The script makes the ambiguity <i>visible</i> before any AI code exists.
5	Evaluation set authored and frozen. ~15 questions, expected tables, expected paths, committed.	DR-16. It sits here because until step 3 the true paths could not be known, and after step 6 the set could no longer be authored innocently (T-04).
6	Retrieval. Embeddings, hybrid scoring, anchors, expansion, both bounds — tuned against step 5.	Ground truth exists before tuning begins.
7	Generation, validation, execution. Prompt, model adapter, sqlglot, conformance, read-only run. First real answer.	The smoke check becomes a genuine end-to-end test here.
8	Trace assembled, persisted, served. Migrations, JSONB plus extracted columns, API returns it. <code>user_id</code> present from this first migration.	Identity in the schema now; the authentication flow much later.
9	Frontend. Ask, result, trace panel, graph with highlighted path.	R-05 — the visual centrepiece, built with time remaining, not squeezed at the end.
10	Evaluation run in both modes, result recorded in the README.	Criterion 6. The one measured sentence.
11	Authentication. Registration, login, sessions, history.	Deliberately after the pipeline works. T-08's cookies-across-origins afternoon is far cheaper against a working system, and the measured result exists before a day is spent on plumbing.
12	Deployment, including the global spend ceiling.	The ceiling ships <i>with</i> the public URL, never after it. A public endpoint with an unmetered key is a stranger's budget to spend.

#	DELIVERABLE	WHY IT SITS HERE
13	README and demo video.	Criteria 10 and 11.
14	The supporting layer. Caching, rate limiting, example questions, re-ingest endpoint.	The only item behind which no criterion sits. If anything gives, it gives here — deliberately placed last so that decision is already made.

HONEST NOTE ON THE TAIL

Steps 11, 12 and 13 are all Core. The slack in this sequence is **one item**, not four. "Core before Supporting" is true but thinner than it sounds, and the reordering that put the spend ceiling into deployment and the supporting layer behind the README exists precisely to make that one item unambiguous.

10 The checkpoint ritual

Named deliverable. Run by the owner at every deliverable boundary — not on a clock.

#	STEP	DETAIL
1	Run the smoke check from a clean start	Not from the running development environment. Stop everything, docker compose up, run NFR-27's single command. It exits pass or fail.
2	Run the test suite	Including the negative cases NFR-26 requires.
3	Tag the commit	checkpoint-04-pathfinding and so on. This is the recovery mechanism. After it, nothing subsequent can cost more than the work since this tag.
4	Write three lines in CHECKPOINTS.md	What works now that did not before · what was built that cannot yet be explained ("to dissect") · what the next single deliverable is.

The one rule that matters: if it is not green, the next deliverable does not start. Fix it, or reset to the last tag and redo the step. Building the next thing on top of something broken is precisely how a state becomes unrecoverable, and it is the failure mode the ritual exists to prevent.

Two things this accumulates. The "to dissect" column becomes a list, written fourteen times at the moment each gap was noticed — and that list is the agenda for the dissection day, requiring no reconstruction from memory. And CHECKPOINTS.md in a public repository is a record of deliberate process, which almost no student project carries.

11 Decision index

Every design decision with the option it was chosen against. This table is the README's design-decisions section.

ID	DECISION	REJECTED	THE DECIDING REASON
DD-01	Pure core, impure edges	Conventional service layers owning their own I/O	The components that matter most are pure computation; keeping them pure makes them testable without infrastructure, readable alone, and unable to half-fail.
DD-02	Orchestrator assembles the trace; two connections never mix; one frontend API client	Stages writing their own trace fragments	Makes NFR-13 and NFR-07 structural rather than hopeful. Nothing is written until it is written once.
DD-03	One Postgres instance, two databases, two roles	Two separate instances	The guarantee comes from the role, not from separation, so separation buys nothing it does not already have.
DD-04	Orchestrator owns error handling; stages return outcomes	Per-stage exceptions	Keeps stages pure, keeps error paths in one place, and means a failure still produces a trace.
DD-05	The trace carries its own copy of the subgraph	Storing names and re-rendering against the live schema	Re-ingestion would silently change what old traces depict — the project's own target failure, occurring inside the mechanism built to prevent it.
DD-06	The narrative is composed once and stored	Composing at read time	Keeps explanation logic in one testable backend place and the frontend a renderer of strings. Cost: old traces keep old wording.
DD-07	JSONB body plus five extracted columns	Full normalisation; pure JSONB	A trace is a document — written once, read whole, never joined. The five questions actually asked across traces get real columns.
DD-08	Embed enriched descriptions	Embedding raw identifiers	TPC-DS names are compressed identifiers; "revenue" cannot reach <code>ss_ext_sales_price</code> without them. This is threat T-03, landing on the component the project rests on.
DD-09	Hybrid semantic + literal scoring	Embeddings alone	The two methods fail in different places. Also yields two sub-scores per candidate, so the trace can say <i>why</i> something was retrieved.
DD-10	Expand along the graph from anchors	Passing the top-k scored tables	Without expansion the graph is decoration and the structure does no work. Yields ambiguity detection at no extra cost.
DD-11	Two bounds, protected cap set, no column pruning	A single score-based cap	A blind cap removes connectors and keys — silently and fatally. Curating columns within a table is a hidden editorial decision the trace cannot reveal.

ID	DECISION	REJECTED	THE DECIDING REASON
DD-12	Shortest path; deterministic arbitrary tie-break; optional declared preference	Domain or cardinality heuristics; refusing to answer	Any heuristic here is an assumption about a <i>business</i> , and Cartograph has no business context. A system that appears to choose cleverly invites the user to stop checking.
DD-13	Conformance: four outcomes, flag rather than reject	Exact equality; rejecting divergent SQL	The lie to prevent is a trace disagreeing with the SQL, not the model choosing differently. <code>not_checked</code> prevents manufactured confidence.
DD-14	DDL-format prompt, JSON output, no model self-explanation	Prose or JSON schema rendering; free-text output; asking the model to explain	An unverifiable explanation is worse than none in a project about inspectable reasoning.
DD-15	Retry fixable faults only, capped at two, all attempts recorded	No retry; unconditional retry	Retrying a rejected write operation would mean coaching a model past a safety check.
DD-16	One overlay file, three sections	Three separate files	All three are the same kind of thing: facts the catalog cannot supply.
DD-17	Traces separate; graph as edge rows; snapshots added not replaced	Trace as a column; graph as JSON; snapshots replaced	History listing must not drag document bodies; the trace stays the <i>one</i> document; <code>schema_ref</code> never dangles.
DD-18	Tables as graph nodes, columns listed inside them	Columns as nodes, matching the internal model	Rendering columns as nodes produces the hairball NFR-18 forbids.
DD-19	Trace panel leads with plain English; prominence proportional to ambiguity	A flat panel in pipeline order	A trace that is complete but unread has failed. Burying the warning politely recreates the project's target failure behind a better data structure.
DD-20	The evaluation measures the generated SQL, not the intermediate state	Comparing retrieved tables and selected paths across modes	The originally specified metric does not exist on one side of the comparison — full-schema prompting has no retrieved set and no selected path.
DD-21	The ambiguity warning fires only when the selection was <i>arbitrary</i>	Firing whenever more than one valid path exists	With roughly eighty edges, multiple paths are the common case. A warning that always fires is not a warning. A tie means the rule had no basis; a shortest-path win means it had one.

12 The evaluation harness

Criterion 6 depends on this, and the metric originally specified could not have been computed.

DD-20 • Measure the SQL, not the intermediate state

FR-37 specifies that the evaluation compares *retrieved tables* and *chosen join path* against expectations. That is well-defined for the retrieval pipeline. **It is undefined for the comparison mode.** Full-schema prompting has no retrieval step and no path

selection — there is no retrieved table set and no selected path to compare. As written, the two modes could not be compared at all.

The correction: measure the artefact both modes produce — the SQL. For each question, in each mode, extract the joins the generated SQL actually performs and compare them to the expected path.

THE INSTRUMENT ALREADY EXISTS

Extraction is exactly what the conformance checker does (DD-13). **The safety component doubles as the measurement component** — the same code that verifies the model followed a supplied path can recover the path from SQL produced with no path at all. And because the same instrument measures both sides, its imperfections are symmetric: even where absolute numbers are soft, the *comparison* remains meaningful.

REPORTED	SCOPE	DEFINITION
Path correctness	both modes	Questions whose generated SQL joined along the expected path, as a fraction of questions measured.
Table correctness	both modes	Questions whose generated SQL referenced the expected set of tables.
Retrieval recall / precision	retrieval mode only	Internal diagnostic for tuning. Not part of the headline comparison , because it has no counterpart.
Not measurable	—	Questions whose SQL returned not_checked drop out of the denominator and the count is reported.

Method. Fifteen questions × two modes = thirty model calls per run, comfortably within budget. Temperature 0. **The README states that the figure is a single run at temperature zero** — with fifteen questions, a 13-versus-9 result could partly be chance, and overstating it is precisely what an interviewer catches. Three runs and a mean is better if budget allows; either way the method is stated.

The result may not favour the project, and that must be decided about now rather than then. TPC-DS is roughly twenty-four tables, which fits comfortably in a modern model's context window. Full-schema prompting is therefore not handicapped, and the honest possibility is that the comparison returns even. That is the schema-size threat arriving at the one number intended for a résumé.

A null result is a real finding — *"at twenty-four tables, full-schema prompting matches graph retrieval; the benefit should appear as schema size grows"* — and it is a far better interview answer than a suspiciously favourable number. The commitment is made here, in advance: **whatever it says gets reported.**

Two claims, not one. The **empirical** claim — that retrieval produces more correct join paths — is what this measures, and it may come out even. The **structural** claim — that the system reports when a choice existed — is not a measurement but a property: full-schema prompting has no notion of alternative paths and cannot do this at all. It is the stronger of the two claims and must be stated as a property, never dressed up as a result.

13 Corrections from the coherence pass

Six inconsistencies found by reading the whole design against itself before building. Two would have surfaced during the

build.

CORRECTION 1 • TWO DECISIONS CONTRADICTED EACH OTHER

IR-05 listed **path-conformance failure as an error code**, meaning the query fails and the user receives an error. DD-13 executes anyway and flags. Both appeared in the documents; both could not be right.

Resolved in favour of DD-13. Conformance divergence is a trace annotation, not an error condition. IR-05 loses that entry; its remaining codes are validation failure, execution failure, model failure, and question-not-answerable.

CORRECTION 2 • THE TRACE HAD NOWHERE TO RECORD WHAT ACTUALLY HAPPENED

DD-13 says that on divergence the trace reports the path the SQL *actually* used. The trace shape carried only `paths.selected_id`. With one field, the divergence is invisible: either the selected path is shown and the truth hidden, or the actual path is shown and the fact that anything went wrong is hidden. Criterion 13 requires both.

Added: `paths.actual_edges[]` and `paths.diverged`.

CORRECTION 3 • THE NAMING OVERLAY WAS IN THE WRONG BUILD STEP

Build step 4 delivers path finding *and the Explainer*. Readable descriptions were placed at step 6 with retrieval. The Explainer would therefore have had only raw identifiers and would have emitted "*through store_sales via ss_store_sk*" — exactly the unreadable output criterion 14 exists to prevent. DD-08 already stated that descriptions are built at ingestion; the sequence contradicted it.

Moved to step 3.

CORRECTION 4 • THE BOUND COULD CUT THE TABLES THAT MAKE JOINS WORK

A ten-table cap applied by score can remove a connector table pulled in by expansion, leaving anchors that no longer join. The same failure as cutting key columns, one level up.

Added the protected set (DD-11): anchors and every table on a discovered path between them are exempt from the cap; if the protected set alone exceeds it, the lowest-scoring *anchor* is dropped and expansion re-run.

CORRECTION 5 • THERE WERE TWO BOUNDS, AND ONLY ONE WAS NAMED

FR-10 reads as though a single limit exists. The Retriever bounds how many anchors it proposes; the Pathfinder bounds the expanded subgraph. Different limits, different owners, different tuning.

Both named and assigned (DD-11), and both record what they excluded.

CORRECTION 6 • PATH ENUMERATION COULD BREACH ITS OWN LATENCY BUDGET

Enumerating every path between anchors is trivial at twenty-four tables. At the fifty tables NFR-05 promises to handle, global enumeration of simple paths can grow badly and put NFR-02's one-second budget at risk on exactly the schema size the requirement guarantees.

Constrained: enumeration is pairwise between anchors with the hop limit applied, never global across the subgraph.

CORRECTION 7 • A FIX I RECOMMENDED WAS UNSOUND

Faced with "add roughly thirty foreign keys from the specification", I proposed deriving them programmatically from the `_sk` naming convention. **Verification showed the convention does not hold.** `ss_addr_sk` references `customer_address`, not `"addr"`; `ss_cdemo_sk` references `customer_demographics`; `ss_sold_date_sk` references `date_dim`; `ss_promo_sk` references `promotion`. About half the columns break the pattern, and a script built on it would have produced a **confidently wrong graph** — precisely the failure T-01 describes, introduced by the fix intended to prevent it.

Superseded by DD-10: the toolkit's published `tpcds_ri.sql` makes derivation unnecessary.

CORRECTION 8 • AN ESTIMATE WRONG BY THREEFOLD

The foreign key count was stated as "roughly thirty". It is **roughly eighty**. Consequential in two places: the graph is considerably denser than the design assumed, which makes correction 6's pairwise-with-hop-limit constraint load-bearing rather than precautionary; and multiple valid paths become the common case rather than the exception, which is what forced DD-21.

CORRECTION 9 • A RISK FIX WAS SCOPED TOO NARROWLY, AND THE WAREHOUSE SAID SO

On 7 September, faced with conformance checking being a plausible two-day rabbit hole, the v1 scope was cut to top-level JOIN . . . ON nodes only, with everything else returning `not_checked`. Defensible on the information available.

Reading TPC-DS query q3 during Day Zero disproved the assumption behind it. The benchmark's own queries carry no JOIN keyword — comma-separated FROM, join predicates as equalities in WHERE. The narrow checker would have abstained on the dominant query form in the very schema it was built to check, shrinking the evaluation denominator while every hand-written test passed.

DD-13 amended: v1 extracts WHERE-clause equality predicates between qualified columns alongside ON clauses. Mechanically the same operation; materially different coverage.

Found by reading one query before writing any code. That is the third time in this project that verification before building has been cheaper than discovery during it.

Two observations that needed no change

The subgraph is stored twice — once structured for rendering (DD-05), once as text inside the stored prompt (FR-17). Deliberate: one copy to draw from, one to audit what was actually sent.

Verification before building is what made corrections 7 and 8 cheap. Both were found by reading source material for twenty minutes rather than by discovering them on day three. That is the whole argument for the design phase, arriving as evidence rather than as assertion.

The deployed warehouse may be smaller than the local one if the free-tier storage check at step 2 forces a lower scale factor. Acceptable, but it is stated in the README rather than left to be discovered.

14 Status

PHASE	STATUS	DELIVERABLE
1 • Planning & feasibility	Closed	Project Charter v2 — 15 definition-of-done criteria, 8 decisions, 9 risks.
2 • Requirements	Closed	SRS v2 — 103 requirements, traceability, 8 threats.
3 • Design	Closed	This document — 20 decisions, ordered build sequence, checkpoint ritual.
4 • Build	Next	Fourteen deliverables, \$09. Step 1 is standing the stack up empty; step 2 answers T-01.
5-7 • Test, deploy, maintain	Interleaved	Testing is continuous per NFR-21 and NFR-26; deployment is step 12.

Deliberately deferred to the build: test suite structure — fixtures, what runs without a database, how the negative cases are organised. NFR-26 fixes what must be tested; designing how in the abstract would be guessing.

IMMEDIATE NEXT ACTION

Build step 1: WSL2, Docker, Postgres with pgvector, Redis, and hello-world through every layer, starting from one docker compose up, with the repository public and CI green from the first commit. Then step 2 — which answers, on day one, whether the foundational assumption of this entire document holds.

Revision history

VERSION	DATE	CHANGE
3	12 Sep 2026	DD-13 amended after Day Zero read TPC-DS query q3: the benchmark's queries use implicit joins — comma-separated FROM with predicates in WHERE — so conformance v1 must extract both forms rather than abstaining on the schema's own house style. Correction 9 added . Also recorded: q3 confirms the star shape, with date_dim and item connectable only through store_sales — a concrete instance of why DD-10's expansion exists.
2	7 Sep 2026	Verification pass against the TPC-DS schema and the toolkit's constraint file. Figure 3 confirmed — all three routes from a sale to a region are real. DD-21 added : the ambiguity warning fires only when the selection was arbitrary, since roughly eighty edges make multiple paths the common case. DD-16 revised : the overlay's relationships are generated from the published tpcds_ri.sql rather than authored. Corrections 7 and 8 added , both against recommendations made in version 1.
1	7 Sep 2026	Issued. Twenty design decisions across architecture, the trace object, algorithms, persistence and frontend; the ordered build sequence and checkpoint ritual as named deliverables; six corrections from a whole-design coherence pass; the evaluation metric corrected from intermediate state to generated SQL.